数据结构 A 第 4 次作业

姓名: 王子川　　**学号:** PB25000055

1(3.6) 出栈顺序

一个核心观察是: 任何一个状态下, 栈 S 中的元素从栈底向栈顶一定是递增的.

容易通过归纳法证明. 初始状态栈 S 为空, 显然满足. 若前 k 次操作后栈 S 中的元素从栈底向栈顶递增, 则在第 $k+1$ 次操作时, 若向栈 S 中 `push()` 一个元素, 由于输入序列为 $1, 2, \ldots, n$, 新入栈的元素比之前入栈的所有元素都大, 因此仍保持从栈底向栈顶递增; 若从栈 S 中 `pop()` 栈顶元素, 则剩余元素的相对次序不变, 因此仍然递增.

所以在任意状态下, 栈 S 中的元素从栈底向栈顶都是递增的.

回到原命题, 使用反证法. 假设存在

$$i < j < k, \quad p_j < p_k < p_i.$$

在这三个元素中, p_i 最先出栈. 考虑 p_i 刚刚出栈后的状态.

因为

$$p_j < p_k < p_i,$$

而输入序列为 $1, 2, \ldots, n$, 所以在 p_i 入栈之前, p_j, p_k 都已经入栈. 又因为 $j > i, k > i$, 所以当 p_i 出栈时, p_j, p_k 尚未出栈. 因此, 此时 p_j, p_k 都在栈 S 中.

又因为 $p_j < p_k$, 根据栈中元素从栈底向栈顶递增的性质, p_j 一定在 p_k 的下面. 因此根据栈的 LIFO 原则, p_k 必须先于 p_j 出栈.

但是在输出序列中 $j < k$, 即 p_j 先于 p_k 输出, 矛盾.

因此不存在

$$i < j < k, \quad p_j < p_k < p_i.$$

原命题得证.

2(3.19) 检查括号匹配

将尚未匹配的左括号存入栈 s 中. 从左到右依次扫描表达式, 若遇到左括号, 则将其入栈; 若遇到右括号, 则检查栈是否为空以及栈顶的左括号是否与其匹配. 若栈为空或括号不匹配, 则说明表达式中的括号不能正确匹配; 否则将栈顶元素出栈.

扫描结束后检查栈 s 是否为空. 若栈为空, 则所有括号均正确匹配; 否则说明存在尚未匹配的左括号.

```
bool ParenthesesMatch(SqList L) {
    InitStack(s);
    int n = L.length;

    for (int i = 1; i <= n; i++) {
        char op = L.elem[i];

        if (op == '(' || op == '[' || op == '{') push(s, op);

        else if (op == ')') {
            if (StackEmpty(s) || s.top != '(') return false;
            pop(s);
        }
        else if (op == ']') {
            if (StackEmpty(s) || s.top != '[') return false;
            pop(s);
        }
        else if (op == '}') {
            if (StackEmpty(s) || s.top != '{') return false;
            pop(s);
        }
    }

    return StackEmpty(s);
}
```

设表达式长度为 n. 每个字符至多入栈、出栈各一次, 因此时间复杂度为 $O(n)$, 最坏情况下空间复杂度为 $O(n)$.

3(3.21) 逆波兰表达式

扫描中缀表达式. 若遇到操作数, 直接加入 postfix; 若遇到左括号, 将其入栈; 若遇到右括号, 将栈顶元素依次出栈并加入 postfix, 直到遇到左括号; 若遇到运算符, 则将栈中优先级不低于当前运算符的运算符依次出栈并加入 postfix, 再将当前运算符入栈. 扫描结束后, 将栈中剩余运算符依次加入 postfix.

```
int priority(char ch) {
    if (ch == '+' || ch == '-') return 1;
    if (ch == '*' || ch == '/') return 2;
    return 0;
}

string toPostfix(string expression) {
    InitStack(op);
    string postfix;

    for (char ch: expression) {
        if (ch == ' ') continue;
        else if (isalpha(ch)) postfix += ch;

        else if (ch == '(') push(op, ch);
        else if (ch == ')') {
            while (op.top != '(') {
                postfix += op.top;
                pop(op);
            }
            pop(op);
        }

        else {
            while (!StackEmpty(op) && op.top != '('
                   && priority(op.top) >= priority(ch)) {
                postfix += op.top;
                pop(op);
            }
            push(op, ch);
        }
    }

    while (!StackEmpty(op)) {
        postfix += op.top;
        pop(op);
    }
    return postfix;
}
```

设字符串长度为 n. 每个字符至多入栈、出栈、加入 postfix 各一次, 因此时间复杂度为 $O(n)$, 最坏情况下空间复杂度为 $O(n)$.